

deti
universidade de aveiro
departamento de eletrónica,
telecomunicações e informática

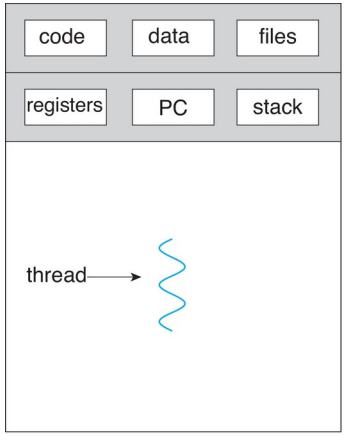
Threads and Concurrency

Real-Time Embedded Systems (#41505)

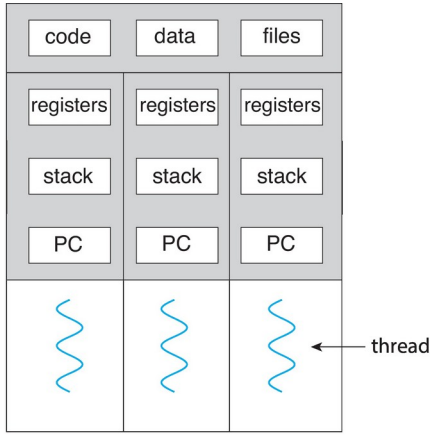
Paulo Pedreiras, pbrp@ua.pt
Department of Electronics, Telecommunications and Informatics
University of Aveiro
Academic year 2026/2027

Outline

- Overview
- Multi-core Programming
- Multi-threading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples



single-threaded process



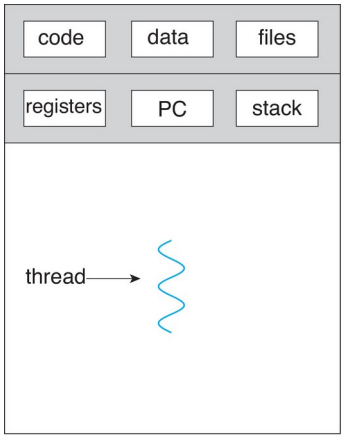
multithreaded process

Motivation

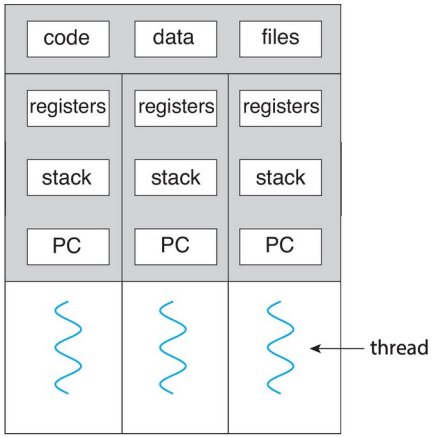
- Most modern applications are multi-threaded
- Threads run within application and allow to carry out several tasks in parallel:

  E.g.

  - Update display
  - Fetch data
  - Spell checking
  - Answer a network request



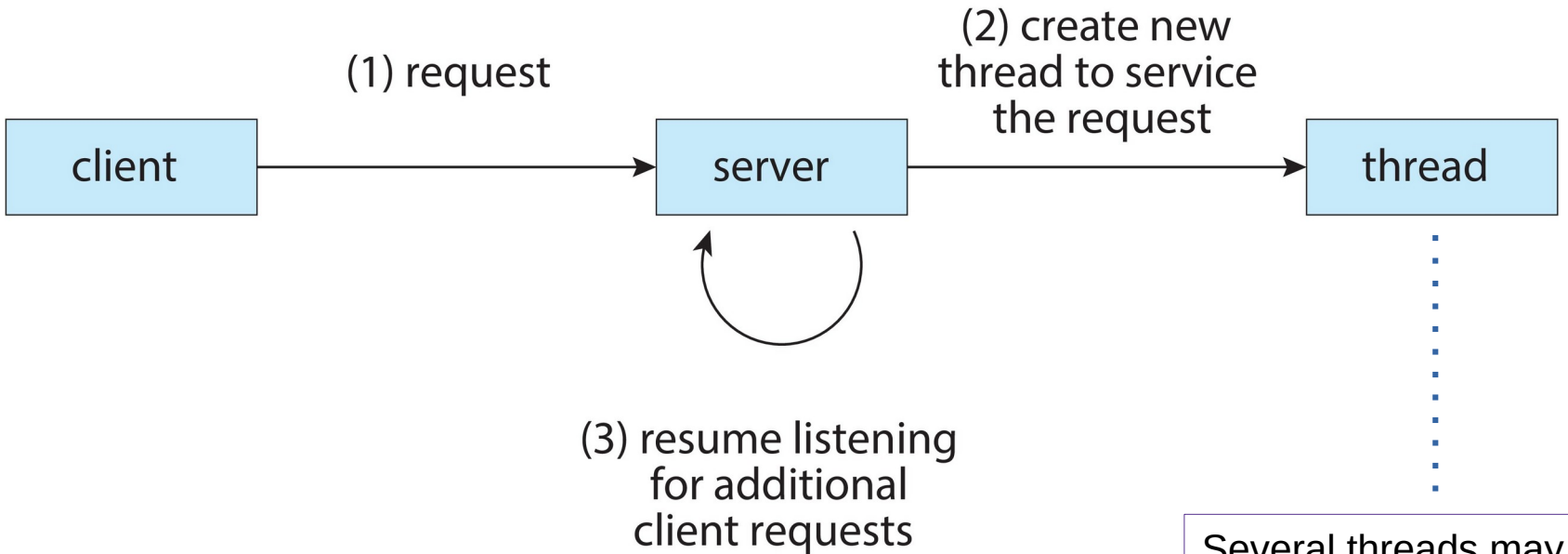
single-threaded process



multithreaded process

- Process creation is “heavy-weight” while thread creation is “light-weight” (thanks e.g. to the shared address space)
- Can simplify code, increase efficiency and throughput
- Modern kernels are generally multi-threaded

E.g.: multi-threaded server



Several threads may run “at the same time”, one for each client

Benefits

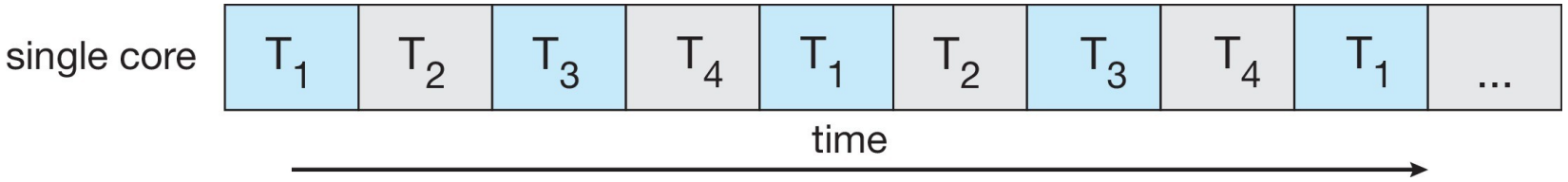
- **Responsiveness –** may allow continued execution if part of process is blocked, especially important for user interfaces
- **Resource Sharing –** threads share resources of the main process (specially the address space), turning communication and synchronization among threads easier than shared memory or message passing
- **Economy – "**cheaper" than process creation, thread switching has lower overhead than context switching
- **Scalability –** process can take advantage of multi-core architectures
  - Important performance gains can be obtained e.g. by splitting data in several blocks and create tasks for processing each block in parallel.

Multicore Programming

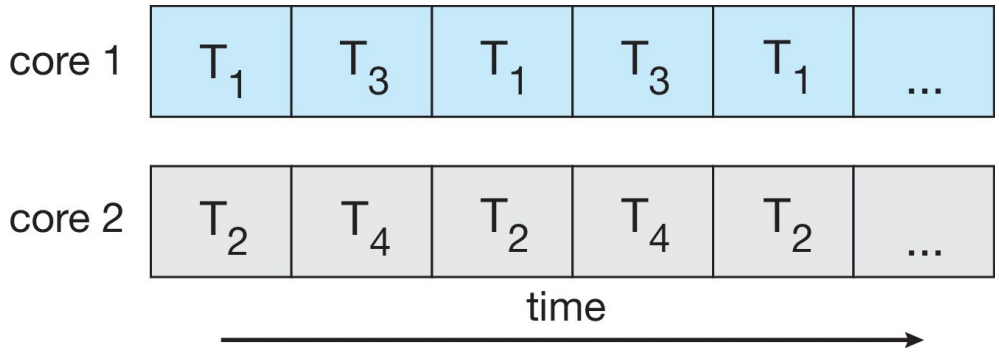
- **Multicore** or **multiprocessors** allow to speed up applications, but programming on such environments adds challenges, like:
  - **Dividing activities in blocks that can be run in parallel**
  - **Data splitting**
  - **Data dependency**
  - **Synchronization**
  - **Load Balance**
  - **Testing and debugging**
  - **...**
- ***Note that:***
  - ***Parallelism*** implies a system can perform more than one task **simultaneously – at the (exact) same time**
  - ***Concurrency*** means that a system allows more than one task making progress. E.g. in a single processor via time-sharing

Concurrency vs. Parallelism

- **Concurrent execution on single-core system:**

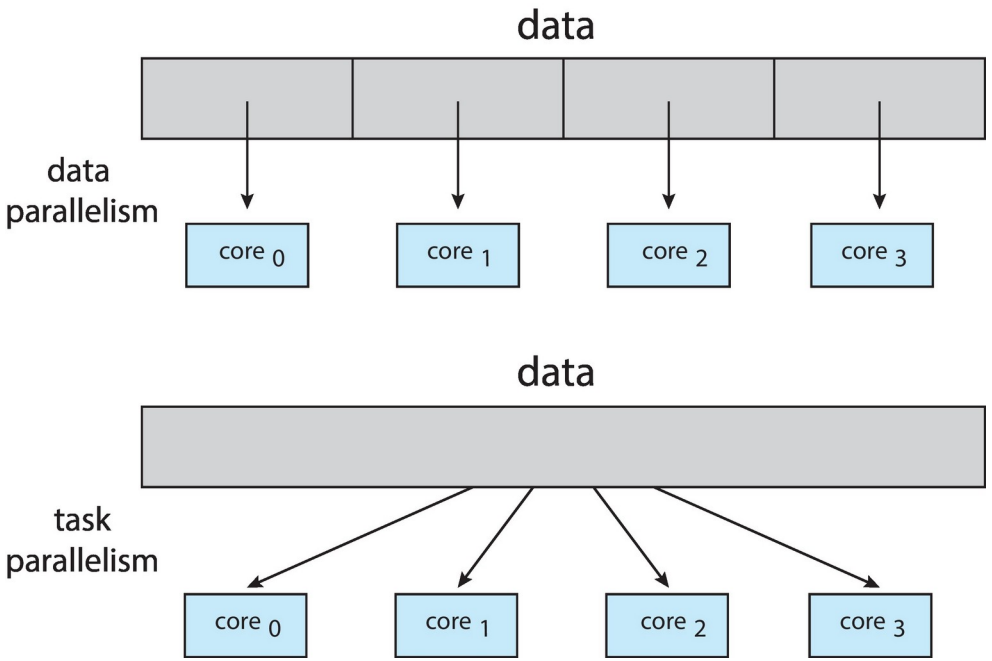


- **Parallelism on a multi-core system:**



Multicore Programming

- Types of parallelism
  - **Data parallelism** – distributes subsets of the same data across multiple cores, same operation on each
  - **Task parallelism** – distributing threads across cores, each thread performing a unique/distinct operation



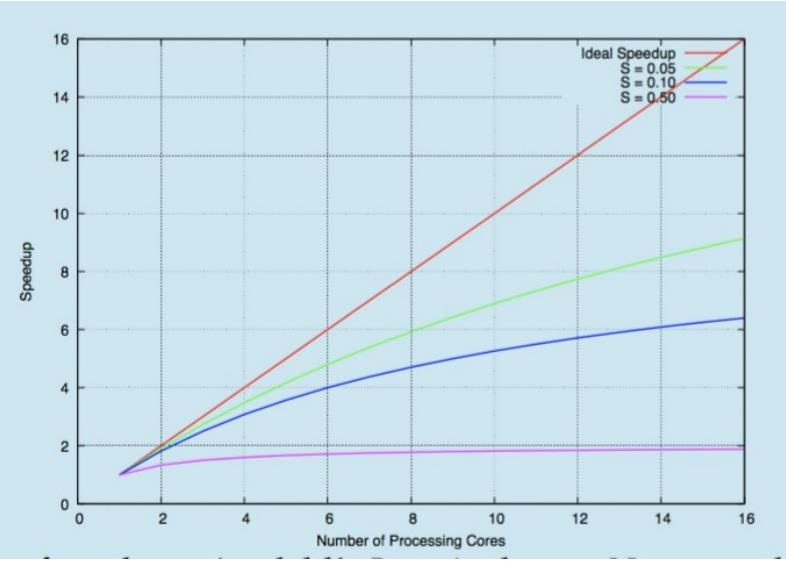
Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- *S* is serial portion
- *N* processing cores

$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if an application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As *N* approaches infinity, speedup approaches 1 / *S*

Serial portion of an application has disproportionate effect on performance gained by adding additional cores



User Threads and Kernel Threads

- **User threads** - management done by user-level threads library
- **Kernel threads** - Supported by the Kernel
- Three primary thread libraries:
  - POSIX **Pthreads**
  - Windows threads
  - Java threads
- Virtually all general-purpose operating systems support threads, including:
  - Windows
  - Linux
  - Mac OS X
  - iOS
  - Android
- Most real-time OSs are centered on the concept of threads/tasks

Thread Libraries

- **Thread libraries** provide programmers with API for creating and managing threads
- Two primary ways of implementing:
  - Library entirely in user space
  - Kernel-level library supported by the OS

Pthreads

- May be provided either as user-level or kernel-level
- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
  - ***Specification***, not ***implementation***
  - API specifies behavior of the thread library, implementation is up to development of the library
  - Common in UNIX operating systems (Linux & Mac OS X)

Linux Pthreads

- Standard API for multithreading in C/C++
- Each thread runs within the same process, sharing memory and resources
- Key functions:

  pthread_create() – start a new thread

  pthread_join() – wait for a thread to finish

  pthread_exit() – thread termination

  pthread_mutex_* – mutual exclusion locks

  pthread_cond_* – condition variables for synchronization

- Enables parallelism and efficient CPU utilization
- Allows the Fork–Join model:

  Fork: main thread creates worker threads (pthread_create())

  Join: main thread waits for workers to finish (pthread_join())

Linux Pthreads Example

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
   pthread_t tid; /* the thread identifier */
   pthread_attr_t attr; /* set of thread attributes */

   /* set the default attributes of the thread */
   pthread_attr_init(&attr);
   /* create the thread */
   pthread_create(&tid, &attr, runner, argv[1]);
   /* wait for the thread to exit */
   pthread_join(tid,NULL);

   printf("sum = %d\n",sum);
}
```

Linux Pthreads Example (Cont.)

```
/* The thread will execute in this function */
void *runner(void *param)
{
   int i, upper = atoi(param);
   sum = 0;

   for (i = 1; i <= upper; i++)
      sum += i;

   pthread_exit(0);
}
```

Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
   pthread_join(workers[i], NULL);
```

Implicit Threading

- Design, debug and validation of multithreaded programs is much harder than single-threaded ones.

  Such difficulty increases with the number of threads used
- Implicit creation and management of threads done by compilers and run-time libraries rather than programmers is gaining popularity
- Several approaches available
  - Thread Pools
  - Fork-Join
  - OpenMP
  - Intel Threading Building Blocks

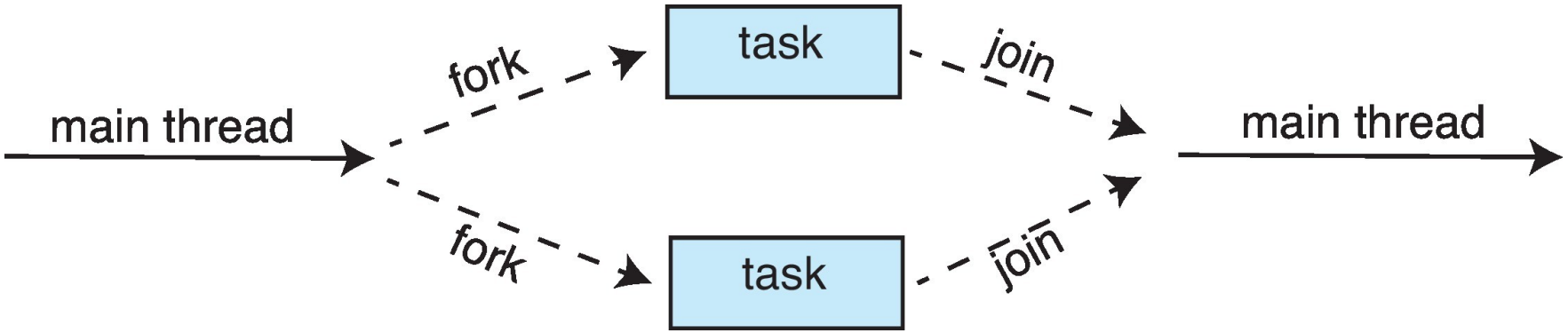
Thread Pools

- Create a number of threads at start-up, and place them in a pool where they await work
- Advantages:
  - Usually slightly faster to service a request with an existing thread than create a new one
  - Allows the number of threads in the application(s) to be bound to the size of the pool
  - Separating the task to be performed from the mechanics of creating task allows different strategies for running tasks
    - i.e,Tasks could be scheduled to run periodically
- Windows API supports thread pools:

```
DWORD WINAPI PoolFunction(AVOID Param) {
    /*
    * this function runs as a separate thread.
    */
}
```

Fork-Join Parallelism

- Multiple threads (tasks) are **forked**, and then **joined**.



Fork-Join Parallelism

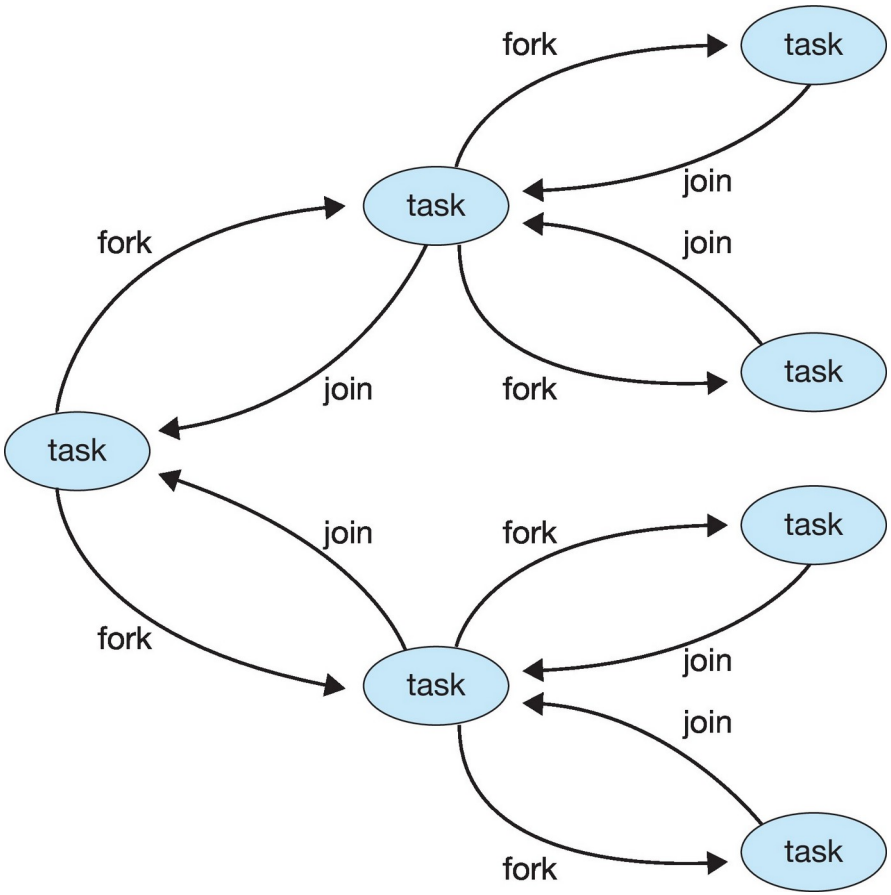
- General algorithm for fork-join strategy:

```
Task(problem)
   if problem is small enough
     solve the problem directly
   else
     subtask1 = fork(new Task(subset of problem)
     subtask2 = fork(new Task(subset of problem)

     result1 = join(subtask1)
     result2 = join(subtask2)

     return combined results
```

Fork-Join Parallelism



OpenMP

- Set of compiler directives and an API for C, C++, FORTRAN
- Provides support for parallel programming in shared-memory environments
- Identifies **parallel regions** – blocks of code that can run in parallel
- E.g. **`#pragma omp parallel for`**

  parallelizes a for-loop so that its iterations are divided among multiple threads that run concurrently.

  Implicit barrier at end (for synch)

```
#include <omp.h>
#include <stdio.h>

int main(int argc, char *argv[])
{
  /* sequential code */

  #pragma omp parallel for
  for (i = 0; i < N; i++) {
    c[i] = a[i] + b[i];
  }

  /* sequential code */

  return 0;
}
```

Intel Threading Building Blocks (TBB)

- Template library for designing parallel C++ programs
- Example: parallel_for()

  Splits iteration space into chunks - runs in parallel

  Programmer writes what to do, TBB decides how

```
#include <tbb/parallel_for.h>
#include <iostream>

int main() {
    // Sequential loop
    for (int i = 0; i < 10; i++)
        std::cout << i << " ";

    // Parallel loop
    tbb::parallel_for(0, 10, [ ](int i) {
        std::cout << i << " ";
    });
}
```

Threading Issues

- Semantics of **fork()** and **exec()** system calls
- Signal handling
  - Synchronous and asynchronous
- Thread cancellation of target thread
  - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations

Lets take a closer look at these issues ...

Semantics of fork() and exec()

- Does **`fork()`** duplicate only the calling thread or all threads?
  - Some UNIXes have two versions of fork
- **`exec()`** usually works as normal – replace the running process including all threads

Signal Handling

- **Signals** are used in UNIX systems to notify a process that a particular event has occurred.
- A **signal handler** is used to process signals
  1. Signal is generated by particular event
  2. Signal is delivered to a process
  3. Signal is handled by one of two signal handlers:
     1. default
     2. user-defined
- Every signal has a **default handler** that the kernel runs when handling a signal
  - **User-defined signal handler** can override default
  - For single-threaded, signal delivered to process

Signal Handling (Cont.)

- Where should a signal be delivered for a multi-threaded process?
  - Deliver the signal to the thread to which the signal applies
  - Deliver the signal to every thread in the process
  - Deliver the signal to certain threads in the process
  - Assign a specific thread to receive all signals for the process

Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is the **target thread**
- Two general approaches:
  - **Asynchronous cancellation** terminates the target thread immediately
  - **Deferred cancellation** allows the target thread to periodically check if it should be canceled
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

  . . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid,NULL);
```

Thread Cancellation (Cont.)

- Invoking thread cancellation requests cancellation, but actual cancellation depends on thread state

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

- If thread has cancellation disabled, cancellation remains pending until thread enables it
- Default type is deferred
  - Cancellation only occurs when thread reaches a **cancellation point**
    - i.e., **`pthread_testcancel()`**
    - Then a **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals

Thread-Local Storage

Thread-local storage (**TLS**) allows each thread to have its instance of data (e.g., buffers, counters)

- Useful when you do not have control over the thread creation process (i.e., when using a thread pool)
- Different from local variables
  - Local variables are visible only during single function invocation
  - TLS is visible across function invocations
- Similar to **`static`** data
  - TLS is unique to each thread
- Most modern thread libraries support TLS
  - E.g. the gcc compiler provides the storage class keyword "__thread" for declaring TLS data. For example, if we want to assign a unique identifier for each thread, we would declare it as:
    - static __thread int threadID;

Thread-Local Storage

- Example - each thread has its own counter

```
#include <stdio.h>
#include <pthread.h>

__thread int counter = 0;  // thread local storage variable

void* work(void* arg) {
    counter++;
    printf("Thread %ld: counter = %d\n", (long)arg, counter);
    return NULL;
}

int main() {
    pthread_t t1, t2;
    pthread_create(&t1, NULL, work, (void*)1);
    pthread_create(&t2, NULL, work, (void*)2);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);
}
```

Operating System Examples
Linux threads

Linux Threads

- Linux refers to them as ***tasks*** rather than ***threads***
- Thread creation is done through the **`clone()`** system call
- clone() creates a new ("child") process, in a manner similar to fork(), but provides more precise control over what pieces of execution context are shared between the calling process and the child process.

  For example, using these system calls, the caller can control whether or not the two processes share the virtual address space, the table of file descriptors, and the table of signal handlers.

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

pthreads are the more common (user-space) method to implement multithreading in Linux: portable, high-level, simpler to use.
Underneath, Linux implements threads with the clone() system call.



- Questions?

Acknowledgment:

This presentation is based on the companion materials provided with the book

Operating System Concepts – 10h Edition, by Silberschatz, Galvin and Gagne